

T E C H C R A F T

R A G

Retrieval-Augmented Generation

Deep-Dive Study Material | Beginner to Advanced | Interview-Ready

Coverage

Complete Theoretical Explanations with Diagrams

Step-by-Step Code Walkthroughs with Comments

Real-World Use Cases & Industry Applications

Practice Tasks & Assignments (Beginner → Advanced)

Interview Questions with Detailed Answers

Quick Revision Cheat Sheet at the End

TECH-CRAFT
— Crafting Skills. Creating Futures —

1. Introduction to RAG

1.1 What is RAG? — Definition

RAG stands for Retrieval-Augmented Generation. It is a powerful AI architecture that enhances the capabilities of Large Language Models (LLMs) by combining two core processes: Information Retrieval and Text Generation.

In simple terms: instead of relying solely on what the model learned during training (which has a cutoff date and limited scope), RAG first retrieves relevant, real-time information from an external knowledge base, and then passes that information to the LLM to generate an informed, accurate answer.

Definition (Official)

RAG is a technique that retrieves relevant documents from an external knowledge base and provides them as context to a language model during inference, enabling the model to generate factually grounded, up-to-date responses.

Analogy: Open-Book Exam vs Closed-Book Exam

Think of a standard LLM as a student taking a closed-book exam — they can only use what they memorized during their studies (training). A RAG-powered LLM is like a student taking an open-book exam — they can look up relevant information in reference books (the knowledge base) before writing their answer. This makes their answers far more accurate, current, and reliable.

1.2 Why Was RAG Created? — The Problem

Before RAG, AI systems faced several critical limitations:

- **Knowledge Cutoff Problem:** LLMs are trained on data up to a specific date. After training, they have no knowledge of new events, updated policies, or recent data.
- **Hallucination:** Without grounding in facts, LLMs tend to make up plausible-sounding but incorrect answers — a phenomenon called hallucination.
- **Private Data Access:** LLMs trained on public data cannot answer questions about internal company documents, private databases, or user-specific data.
- **Expensive Retraining:** Updating model knowledge requires expensive and time-consuming retraining runs.
- **Context Window Limits:** You cannot dump entire databases into an LLM prompt — context windows have limits.

1.3 How RAG Solves These Problems

Problem	How RAG Solves It
Knowledge Cutoff	RAG retrieves from live or recently updated external databases
Hallucination	Grounds LLM responses in retrieved factual documents
Private Data	Connects to your internal/private vector databases
Expensive Retraining	Update the knowledge base — no model retraining needed
Context Limits	Retrieves only the most relevant chunks, not everything

1.4 Real-World Use Cases of RAG

Industry	Use Case	Example
Legal	Contract Q&A	Ask AI about a specific clause in your legal docs
Healthcare	Medical Knowledge Base	Query clinical guidelines & patient records
Finance	Policy & Compliance Bot	Answer questions about regulatory updates
Education	Study Assistant	Ask questions from uploaded textbooks
E-Commerce	Product Support Bot	Answer queries from product manuals & FAQs
HR Tech	Employee Handbook Bot	Employees ask HR policy questions instantly
Customer Support	Intelligent Helpdesk	Route and answer tickets from knowledge base

— Crafting Skills. Creating Futures —

2. Embeddings — The Core of RAG

2.1 What are Embeddings?

An embedding is a numerical vector (an array of floating point numbers) that represents the semantic meaning of a piece of text. The key insight is that text with similar meaning produces similar vectors, and text with different meaning produces vectors that are far apart in mathematical space.

Definition

An embedding is a dense, fixed-size numerical vector that encodes the semantic meaning of text into a high-dimensional mathematical space, such that semantically similar texts have vectors that are geometrically close to each other.

Example: Text to Vector

Consider these sentences:

```
"The cat sat on the mat." → [0.23, -0.41, 0.87, 0.12, ...] (768 or 1536 dimensions)
"A feline rested on the rug." → [0.25, -0.38, 0.84, 0.14, ...] (very similar vector!)
"The stock market crashed." → [-0.71, 0.62, -0.13, 0.95, ...] (very different vector)
```

Notice how sentences about similar topics (cats sitting) produce similar vectors, while an unrelated sentence (stock market) produces a very different vector. This is the power of embeddings.

— Crafting Skills. Creating Futures —

2.2 How Embeddings Are Created

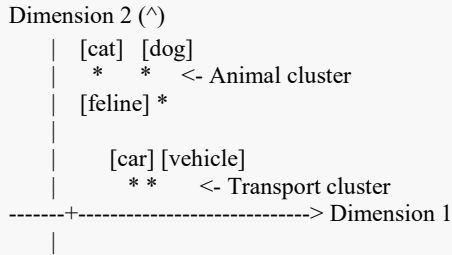
1. Raw text is tokenized (broken into tokens/subwords)
2. Tokens are passed through an embedding neural network model
3. The model outputs a dense vector of fixed dimensions (e.g., 768, 1536, 3072)
4. This vector mathematically encodes the semantic meaning of the input

2.3 Vector Space Diagram

The following diagram shows how words/phrases with similar meanings cluster together in vector space:

VECTOR SPACE VISUALIZATION

High-Dimensional Embedding Space



KEY INSIGHT: Semantically similar words cluster together in vector space.

Similarity Metric: $\text{cosine_similarity}(v1, v2) = (v1 \cdot v2) / (|v1| \times |v2|)$

Range: -1 (opposite) to +1 (identical). RAG retrieves top-K highest similarity chunks.

2.4 Similarity Measurement — Cosine Similarity

To find how similar two embeddings are, we use Cosine Similarity:

$$\text{cosine_similarity}(A, B) = (A \cdot B) / (|A| \times |B|)$$

Where:

$A \cdot B$ = dot product of vectors A and B

$|A|$ = magnitude (length) of vector A

$|B|$ = magnitude (length) of vector B

Result range:

+1.0 → Vectors are identical (same meaning)

0.0 → Vectors are perpendicular (unrelated)

-1.0 → Vectors are opposite (antonyms)

2.5 Popular Embedding Models

Model	Provider	Dimensions	Best For
text-embedding-ada-002	OpenAI	1536	General purpose, widely used
text-embedding-3-small	OpenAI	1536	Cost-effective, faster
text-embedding-3-large	OpenAI	3072	Higher accuracy tasks
all-MiniLM-L6-v2	HuggingFace	384	Free, lightweight, fast
sentence-transformers	HuggingFace	768	Open-source, customizable
embed-english-v3.0	Cohere	1024	Multilingual support

3. Vector Databases

3.1 What is a Vector Database?

A vector database is a specialized database designed to store, index, and efficiently search high-dimensional vector embeddings. Unlike traditional databases that search by exact matches or range conditions (e.g., WHERE age > 30), vector databases search by semantic similarity — finding the vectors that are mathematically closest to a query vector.

Key Point

Traditional DB: Find rows WHERE name = 'John' (exact match) Vector DB: Find documents SIMILAR TO 'tell me about payment policies' (semantic match)

3.2 How Vector Databases Work Internally

5. All document embeddings are indexed using specialized data structures (e.g., HNSW — Hierarchical Navigable Small World graphs, or IVF — Inverted File Index)
6. At query time, the query embedding is compared against stored vectors using Approximate Nearest Neighbor (ANN) algorithms
7. ANN finds the closest vectors without exhaustively checking all stored vectors, making it extremely fast even with millions of embeddings
8. The top-K most similar documents are returned

3.3 Popular Vector Databases — Detailed Comparison

Database	Type	Best For	Hosting	Key Feature
ChromaDB	Open Source	Local dev, prototyping	Self-hosted	Easiest to get started, in-memory or persistent
FAISS	Open Source Library	Research, high speed	Local only	Facebook AI's ultra-fast similarity search library
Pinecone	Managed Cloud	Production at scale	Cloud SaaS	Fully managed, auto-scaling, zero infra overhead
Weaviate	Open Source / Cloud	Hybrid search	Both	Combines vector + keyword (BM25) search

Database	Type	Best For	Hosting	Key Feature
Qdrant	Open Source / Cloud	Advanced filtering	Both	Fast, payload filtering alongside vectors
MongoDB Atlas	Cloud + Vector	Existing Mongo users	Cloud	Integrates vector search into existing MongoDB
pgvector	Postgres Extension	SQL + Vector together	Self-hosted	Add vector search to existing PostgreSQL DB

3.4 Vector DB Operations Explained

INSERT — Storing an Embedding

```
# Pseudocode for storing a document chunk
collection.add(
  ids=["doc_001"],           # Unique ID for this chunk
  documents=["RAG retrieves data..."], # Original text chunk
  embeddings=[[0.23, -0.41, 0.87, ...]], # Pre-computed vector
  metadatas=[{"source": "policy.pdf", "page": 3}] # Optional metadata
)
```

QUERY — Searching Similar Vectors

```
# Pseudocode for similarity search
results = collection.query(
  query_embeddings=[[0.25, -0.38, 0.84, ...]], # Query vector
  n_results=5,                               # Return top 5 matches
  where={"source": "policy.pdf"}              # Optional filter
)
```

4. RAG Architecture & Workflow

4.1 The Complete RAG Workflow — Step by Step

RAG operates in two distinct phases: an offline indexing phase and an online retrieval and generation phase.

#	Stage	What Happens
1	User Query	User types a natural language question (e.g., 'What is the refund policy?')
2	Query Embedding	The question is converted into a numerical vector (embedding) using an embedding model
3	Vector Search	The vector DB searches for the most similar document chunks using cosine similarity
4	Context Retrieval	Top-K relevant chunks (documents) are retrieved from the vector database
5	Augmented Prompt	Retrieved context is combined with the user query to form an augmented prompt
6	LLM Generation	The LLM reads the augmented prompt and generates a context-aware answer
7	Final Answer	The accurate, grounded response is returned to the user

4.2 The Full RAG Pipeline — Both Phases

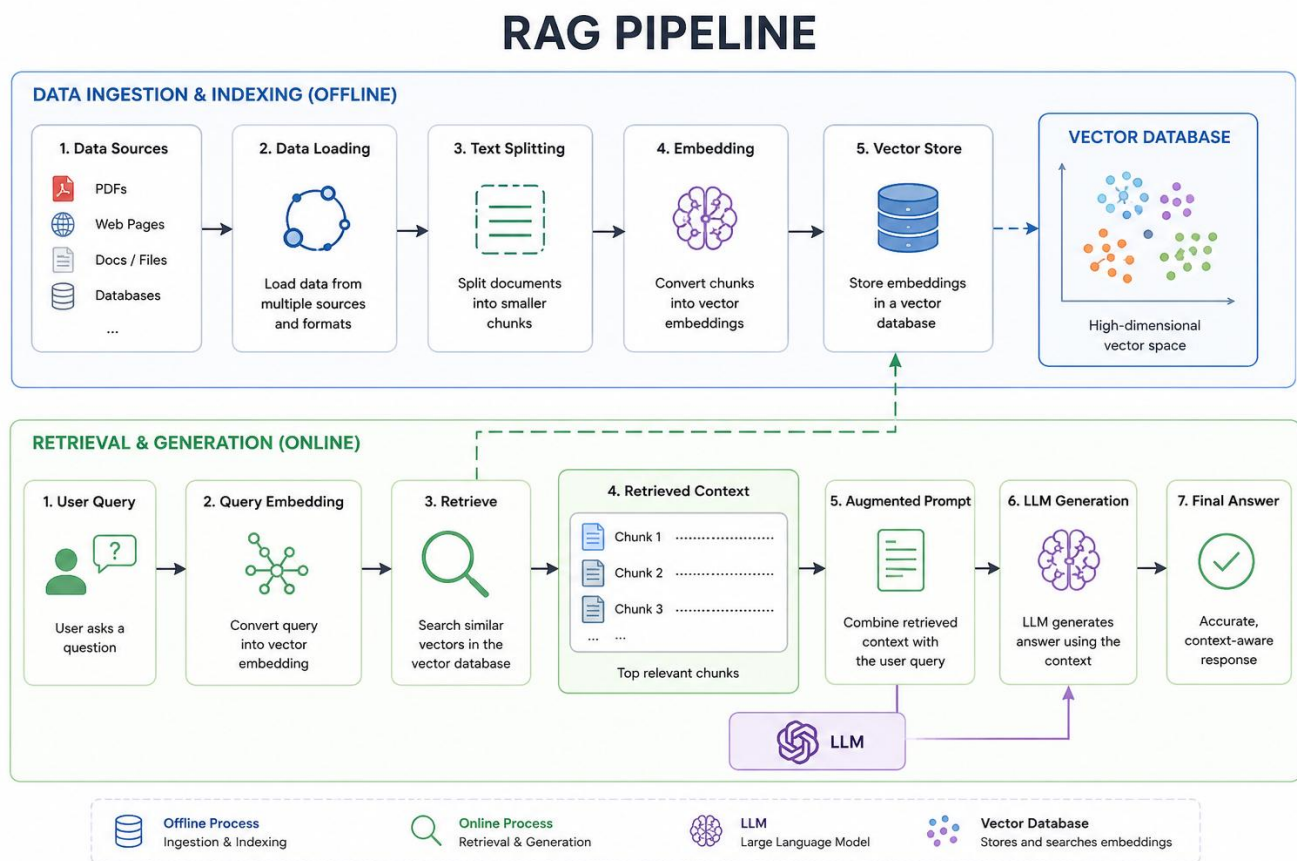
The following diagram illustrates both the offline (data ingestion) and online (retrieval and generation) phases of a complete RAG system:

PHASE 1: DATA INGESTION & INDEXING (OFFLINE)	
1. Data Sources	PDFs, Web Pages, Docs, Databases, APIs
2. Data Loading	Load & parse from multiple formats using loaders
3. Text Splitting	Chunk large docs into smaller overlapping pieces
4. Embedding	Convert each chunk to a vector via embedding model
5. Vector Store	Store all vectors in a vector database for retrieval
PHASE 2: RETRIEVAL & GENERATION (ONLINE)	

1. User Query	User asks a natural language question
2. Query Embedding	Query → vector using same embedding model
3. Retrieve	Search DB for top-K similar vectors
4. Retrieved Context	Fetch the matching text chunks
5. Augmented Prompt	Combine query + context into LLM prompt
6. LLM Generation	LLM reads context and generates answer
7. Final Answer	Return accurate, grounded response

RAG Pipeline Diagram

The diagram below illustrates the complete RAG pipeline, divided into two phases: the offline indexing phase (where documents are processed and stored) and the online retrieval and generation phase (where user queries are answered in real time).



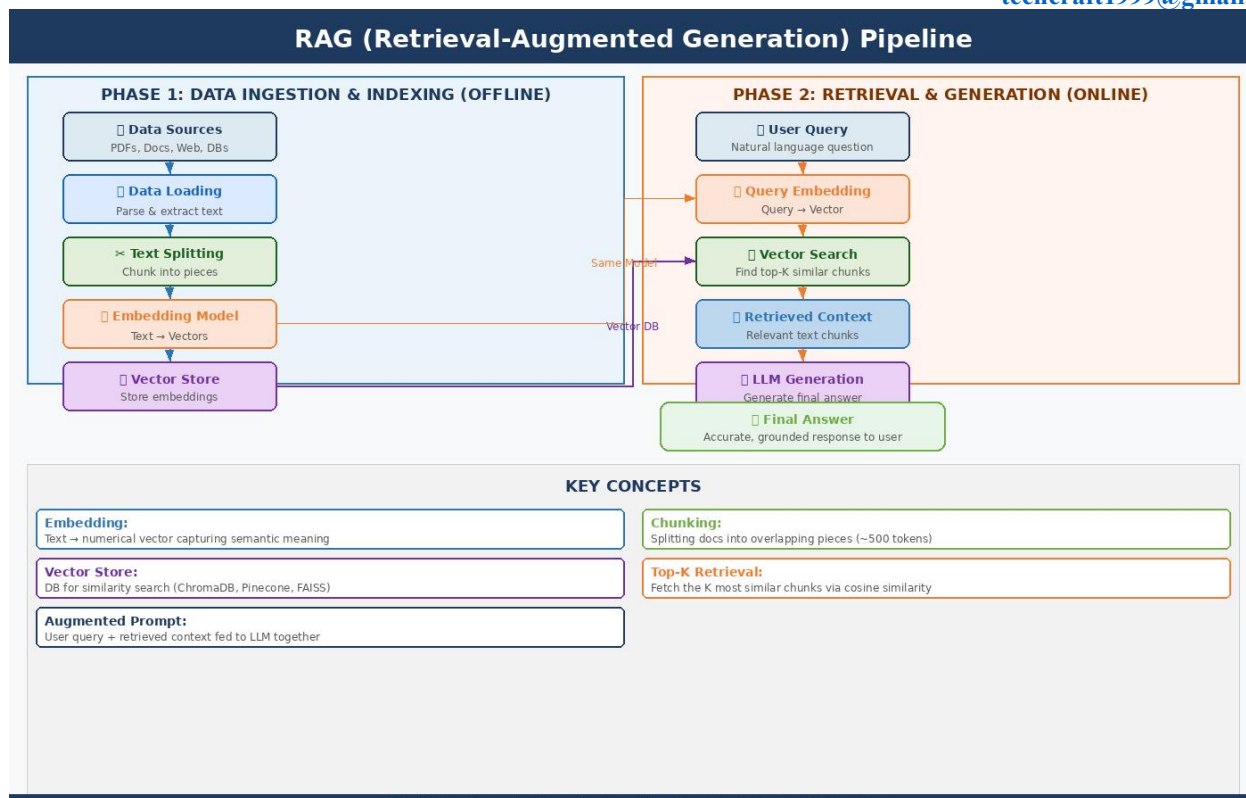


Figure: Complete RAG Pipeline — Offline Indexing Phase (left) and Online Retrieval & Generation Phase (right)

Diagram Explanation

Phase 1 — Data Ingestion & Indexing (Offline, Left Side)

This phase runs once (or periodically when data is updated) to prepare your knowledge base. It consists of five sequential steps: **(1) Data Sources** — Raw documents (PDFs, web pages, databases, APIs) are collected. **(2) Data Loading** — Documents are parsed and converted to plain text. **(3) Text Splitting** — Large text is divided into smaller, overlapping chunks (~500 tokens each). **(4) Embedding Model** — Each chunk is converted to a numerical vector using an embedding model (e.g., OpenAI text-embedding-3-small). **(5) Vector Store** — All vectors are stored in a vector database (ChromaDB, Pinecone, FAISS) for fast similarity search.

Phase 2 — Retrieval & Generation (Online, Right Side)

This phase runs in real time whenever a user asks a question. It consists of five steps: **(1) User Query** — The user types a natural language question. **(2) Query Embedding** — The query is converted to a vector using the **same embedding model** used in Phase 1 (critical for compatibility). **(3) Vector Search** — The vector DB finds the top-K most similar chunks using cosine similarity. **(4) Retrieved Context** — The matching text chunks are fetched and combined into a context block. **(5) LLM Generation** — The LLM receives an augmented prompt (query + context) and generates a factually grounded, accurate final answer.

Key Connection Between Phases

Two critical links connect the two phases. First, the **same embedding model** must be used in both phases — if you indexed documents with OpenAI’s model, you must also embed queries with the same model, otherwise

vectors are incompatible. Second, the **Vector Store** built in Phase 1 is the bridge that the retrieval step in Phase 2 queries in real time. These two shared components are what make RAG a coherent, end-to-end system.

4.3 Offline Phase — Data Ingestion & Indexing

This phase runs once (or periodically when data is updated). It prepares your knowledge base so that the online phase can search it efficiently.

Step 1: Data Sources

RAG can ingest from virtually any data source:

- PDFs — technical manuals, research papers, company policies
- Web Pages — scraped content, documentation sites
- Word Documents / Excel — internal reports, spreadsheets
- Databases — SQL tables exported as text
- APIs — real-time data feeds
- Code Repositories — GitHub codebases for coding assistants

Step 2: Document Loading

Loaders parse different file formats and extract plain text. LangChain provides many pre-built loaders:

```
from langchain.document_loaders import PyPDFLoader, WebBaseLoader

# Load a PDF file
loader = PyPDFLoader("company_policy.pdf")
documents = loader.load()

# Each 'document' object has two parts:
# .page_content -> the actual text on that page
# .metadata      -> info like page number, source file

print(documents[0].page_content) # Text of page 1
print(documents[0].metadata)     # {"source": "company_policy.pdf", "page": 0}
```

Step 3: Text Splitting — Chunking

LLMs and embedding models have token limits. A 500-page PDF cannot be embedded as a whole. We must split documents into smaller overlapping chunks:

```
from langchain.text_splitter import RecursiveCharacterTextSplitter

text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=500, # Each chunk is at most 500 characters
    chunk_overlap=50 # 50 character overlap between adjacent chunks
)

docs = text_splitter.split_documents(documents)

# Example: A 2000-character page might become 4-5 chunks
# Overlap ensures context is not lost at chunk boundaries
```

Why Overlap?

Without overlap: If key information spans the boundary of two chunks, neither chunk contains the full context, and retrieval may fail to find it. With overlap (50 chars): The end of chunk N is repeated as the start of chunk N+1, ensuring boundary information is always captured in at least one chunk.

Chunking Strategy Comparison

Strategy	chunk_size	overlap	Best For
Small Chunks	200-400	20-50	Precise fact retrieval, short answers
Medium Chunks	500-800	50-100	Balanced retrieval — most common
Large Chunks	1000-1500	100-200	Complex reasoning needing more context
Semantic Chunking	Variable	N/A	Split at natural paragraph/section breaks

Step 4: Embedding — Crafting Skills. Creating Futures —

Each text chunk is converted into a vector embedding using an embedding model:

```
from langchain.embeddings import OpenAIEmbeddings
import os

# Initialize the embedding model
embeddings = OpenAIEmbeddings(
    api_key=os.getenv("OPENAI_API_KEY") # Load from environment variable
)

# Embed a single text
vector = embeddings.embed_query("What is the refund policy?")
print(len(vector)) # Output: 1536 (dimensions for ada-002)
print(vector[:5]) # Output: [0.23, -0.41, 0.87, 0.12, -0.55, ...]
```

Step 5: Storing in Vector Database

```
from pymongo import MongoClient

# Connect to MongoDB Atlas (which supports vector search)
client = MongoClient("mongodb+srv://user:pass@cluster.mongodb.net/")
db = client["rag_database"]
collection = db["documents"]

# Store each chunk + its embedding
for doc in docs:
    # Generate embedding for this chunk
    embedding = embeddings.embed_query(doc.page_content)

    # Create the data record
    data = {
        "text": doc.page_content,    # Original text chunk
        "embedding": embedding,      # 1536-dimensional vector
        "metadata": doc.metadata     # Source file, page, etc.
    }

    collection.insert_one(data)

print("All chunks embedded and stored!")
```

TECH-CRAFT
— Crafting Skills. Creating Futures —